

Program Structure and Output

L05 - Program Formatting

Department of Computer Science
University of Pretoria

29 February 2024

- Style is a crucial component of professionalism in software development.
- Clean code that follows stylistic conventions is easier to read, maintain, and share with colleagues.
- Two aspects of clean code include **commenting** code and **indenting** code.

- Comments are included in code to **clarify code** and **give additional information** that cannot be included in the code.
- The principle is rather to write self documenting code than to over comment. It is important to realise that comments **cannot** rectify bad code.
- If code is bad, it needs to be **rewritten**, not commented.

In C++ programming, we have two types of comments. These are:

- **Single-Line comments:** denoted by `//` . These are used when you want to comment on a single line. The `//` tells the compiler to ignore everything after the symbol to the end of the line.

- **Multi-Line comments:** denoted by `/*` and `*/` pair. This type is used when you would like to comment on more than one line. Everything between the two symbols is ignored by the compiler.

Single-Line Comment Example

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     // print welcome message
6     cout << "Welcome to COS132!" << endl;
7     return 0;
8 }
```

- Which line number shows a single-line comment?

Multi-line comment example

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     /* This program will
5         output a welcome message*/
6     cout << "Welcome to COS132!" << endl;
7     return 0;
8 }
```

- Which line number(s) shows a multi-line comment?

Some of the main practices for writing good comments in code are:

1. Every file containing code **must** start with a comment containing the name(s) and student numbers of the author(s), the date of last edit as well as the **purpose** of the file. Use the proper tags for author and date as specified by the documentation generator of your choice.

2. Avoid **redundancy and duplication** of what is already clear in the code.

3. Make sure comments and code agree.

Often programmers change code without updating the accompanying comments. This is unacceptable. *Wrong comments is worse than no comments.* **Use a formal writing style** to state facts in full sentences that are concise and to the point.

4. **Every function** definition **must** be preceded by a comment that briefly describes what the function does. Use the proper tags defined by the documentation generator of your choice to describe all parameters and the return value of the function.

Layout Rules

- Good and consistent code layout enhances code readability.
- If code is readable, it is inherently more understandable and consequently reliable and maintainable.
- We will look at some pointers on layout that need to be taken into account a promote code readability.

- Use **indentation** and **blank** lines to reveal the subordinate nature of blocks of code.
- Each line which is part of the body of a **control structure** (if, while, dowhile, for, switch) is indented one tab stop from the margin of its controlling line.
- The same rule applies to function, struct, or union definitions, and aggregate initialisers.

- **Use blank lines** freely to separate parts of a function or method that are logically distinct.
- Use a blank space around binary operations.
- Leave a blank space after (**and not before**) each comma, colon or semicolon.

- Every line must fit 80 columns.
- We **do not** require specific placement of opening and closing braces. We do, however, require consistency. None of the styles are difficult to understand, choose one of them and use it consistently throughout all the code of one project.

Example Style 1

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     cout << "Programming is great!"<<endl;
8     return 0;
9 }
```

Example Style 2

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     cout << "Programming-is-great!" << endl;
7     return 0;
8 }
```

- What's the difference from Example style 1?

Example Style 3

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     cout<<" Programming - is - great!"<< endl;
8     return 0;
9 }
```

- Use of indentations